ERPLite — Claude Code Master Prompt

Target: VS Code with Claude Code extension (or Antigravity IDE)

Working directory: /home/opengrace/erplite

Agent count: 6 specialised agents (enforced by Claude Code sub-agent calls)

SYSTEM IDENTITY

You are building **ERPLite** — a production-grade, industry-agnostic ERP system modelled on SAP EWM principles, starting with the WM and IM modules. This is not a prototype. Every decision must be made with a 10-year production horizon in mind.

The system is a **modular monolith** (NOT microservices). All modules share one PostgreSQL database and one Go binary. The codebase must be structured so that it can be decomposed into services in the future without rewriting domain logic.

Working directory: `/home/opengrace/erplite`

AGENT DEPLOYMENT INSTRUCTIONS

You MUST spawn the following sub-agents. Each agent has a strictly bounded responsibility. No agent may cross into another's domain. Agents work in parallel where dependencies allow. Use `claude` tool calls for sub-agent delegation.

Agent 1 — Schema Agent

Responsibility: Design and implement the entire PostgreSQL schema. No application code. Schema only.

Rules:

- Every stock change is an **event** appended to `inventory_events`. Stock balances are computed views, never mutable columns.
- The primary entity is `handling_units` (HU). HUs have a `parent_hu_id` for split/merge lineage. This is non-negotiable.
- Use PostgreSQL row-level security for multi-tenant isolation (one `tenant_id` on every table).
- Use `GENERATED ALWAYS AS IDENTITY` for all PKs (not serial, not UUID as PK — use UUID as a secondary `public_id` column for external exposure).

- Partition `inventory_events` by `created_at` (monthly range partitions).
- All foreign keys must have explicit `ON DELETE` rules.
- No nullable columns without a documented reason in a SQL comment.

Required tables (minimum — extend as needed):

```
tenants, users, roles, user_roles
products, product_units
batches
handling_units (id, public_id, tenant_id, parent_hu_id, product_id,
                batch_id, quantity, unit, status, location_id,
                label_version, created_at, updated_at)
locations (id, public_id, tenant_id, warehouse_id, zone, aisle,
           bin, capacity, unit, is_active)
inventory_events (id, tenant_id, event_type, hu_id, from_location_id,
                  to_location_id, quantity, unit, actor_user_id,
                  reference_doc, metadata jsonb, created_at)
warehouse_tasks (id, public_id, tenant_id, task_type, status, hu_id,
                 from_location_id, to_location_id, assigned_to,
                 opened_at, confirmed_at)
barcodes (id, code, tenant_id, entity_type, entity_id, is_active,
          created_at)
price_versions (id, product_id, tenant_id, price, currency,
                valid_from, valid_to)
audit_log (id, tenant_id, actor_id, action, entity_type, entity_id,
           payload_before jsonb, payload_after jsonb, created_at)
blocked_operations (id, tenant_id, reason, context jsonb, created_at)
execution_traces (id, trace_id uuid, tenant_id, created_at)
execution_trace_steps (id, trace_id uuid, agent_name, action,
                       status, payload jsonb, step_order int,
                       created_at)
```

Deliverable: `/home/opengrace/erplite/backend/db/migrations/001_init.sql` with seed data:

- 1 tenant: `techlogix`
- 2 products: Fabric Roll (KG), Laminate (IMP)
- 4 locations: RECV-01, STOR-01, STOR-02, PROD-01
- 2 HUs: HU-1001 (315 KG Fabric, STOR-01), HU-1002 (21.16 IMP Laminate, STOR-02)
- 1 admin user: `admin` / bcrypt hash of `admin123`, role: ADMIN

Agent 2 — Domain Agent

Responsibility: Implement all Go domain models, domain events, and the 8 internal agent

packages. No HTTP handlers. No DB queries. Pure domain logic only.

Directory: `/home/opengrace/erplite/backend/internal/`

Go module path: `github.com/opengrace/erplite`

Required packages:

```
internal/domain/
  models/       — Go structs mirroring DB tables (no ORM tags, use sqlc-generated
types)
  events/       — Domain event types (GoodsReceiptPosted, HUSplit, HUMoved, etc.)
  errors/       — Typed domain errors (ErrDuplicateScan, ErrInsufficientStock,
etc.)

internal/agents/
  barcode/      — Decode → Resolve → Validate (returns entity type + entity ID)
  inventory/    — HU lifecycle: create, fetch, split, merge, close
  warehouse/    — Task creation and confirmation: putaway, move, pick
  production/   — Consumption: full and partial (calls inventory.Split internally)
  labeling/     — Label generation with version increment (never creates new HU
identity)
  pricing/      — Effective-date pricing lookup with historical accuracy
  audit/        — Append-only audit log writer
  errorprevention/ — Duplicate scan detection (5-second idempotency window via
Redis),
                     invalid state transitions, constraint enforcement
```

Agent contract (all agents must implement this interface):

```go
```

```go
type Agent interface {
    Name() string
    Execute(ctx context.Context, req AgentRequest) (AgentResult, error)
}

type AgentRequest struct {
    TraceID   uuid.UUID
    TenantID  int64
    ActorID   int64
    Payload   map[string]any
}

type AgentResult struct {
    Status  string // "SUCCESS" | "FAILED" | "BLOCKED"
    Data    map[string]any
    Message string
}
```

Every agent's `Execute` method must append its step to the execution trace stored in `ctx`. Use a context key `CtxTraceKey` for this.

Handling Unit split logic (CRITICAL — must be exact):

```
HU-PARENT (315 KG, STOR-01, status=AVAILABLE)
  → consume 50 KG
HU-PARENT (265 KG, STOR-01, status=AVAILABLE)  ← update in place
HU-CHILD  (50 KG, PROD-01, status=IN_USE)       ← new row, parent_hu_id = HU-
PARENT.id
inventory_events: SPLIT event referencing both HU IDs
```

Identity is NEVER broken. HU-PARENT retains its ID and label version is incremented. HU-CHILD gets a new ID but carries `parent_hu_id`.

Deliverable: All packages compilable via `go build ./...` with no errors. Unit tests required for: `inventory.Split`, `barcode.Resolve`, `errorprevention.CheckDuplicateScan`.

Agent 3 — API Agent

Responsibility: Implement the HTTP layer only. No business logic. Handlers call services; services orchestrate agents.

Framework: **Gin** (`github.com/gin-gonic/gin`) Database queries: **sqlc-generated** (run `sqlc generate` against the schema)

Layered call chain (enforced, no shortcuts):

```
Handler → Service → Agent(s) → Repository → DB
```

Required endpoints:

```
POST   /api/auth/login           → JWT in HTTP-only cookie (15min access, 7d
refresh)
POST   /api/auth/logout
GET    /api/auth/me

POST   /api/scan                 → Barcode + ErrorPrevention + Inventory agents
POST   /api/consume              → Production + Inventory + Labeling + Audit
agents
POST   /api/move                 → Warehouse + Inventory + Audit agents
POST   /api/split                → Production + Labeling + Audit agents

GET    /api/inventory            → Current stock snapshot (computed from events)
GET    /api/hus/:public_id       → Single HU with lineage
GET    /api/hus/:public_id/lineage → Full parent/child tree
GET    /api/locations            → All locations with HU counts
GET    /api/trace/:trace_id      → Full execution trace with steps
GET    /api/audit                → Paginated audit log (admin only)

GET    /ws                       → WebSocket (gorilla/websocket)
                                   Broadcasts: { type: "agent_trace", payload:
AgentStep }
                                   Broadcasts: { type: "inventory_update",
payload: HUSummary }
```

All mutation endpoints must:

1. Authenticate via JWT middleware
2. Validate input against strict Go struct with `binding:"required"` tags
3. Begin a DB transaction before calling services
4. Commit only if all agents succeed
5. Return the full execution trace in the response body
6. Broadcast each agent step over WebSocket as it executes (not after)

Response envelope (all endpoints):

```
json
```

```
{
  "success": true,
  "data": {},
  "trace": {
    "trace_id": "uuid",
    "steps": [
      { "agent": "BarcodeAgent", "action": "RESOLVE", "status": "SUCCESS",
        "timestamp": "ISO8601", "data": {} }
    ]
  },
  "error": null
}
```

WebSocket broadcast (per agent step, emitted immediately):

```json
{ "type": "agent_trace", "payload": { "agent": "...", "action": "...",
  "status": "...", "timestamp": "..." } }
```

Deliverable: All routes registered, server starts on `:8080`, `go test ./internal/api/...` passes.

Agent 4 — Frontend Agent

Responsibility: Build the complete React frontend. No mock data. All data comes from the real API.

Stack: React 19 + TypeScript + Vite + Tailwind CSS (CDN, no build step for Tailwind in dev) + Zustand + TanStack Query v5

Layout: Three-panel, no page navigation for the operator view.

```
┌──────────────────────────────────────────────────────────────────────┐
│  HEADER: ERPLite | Mode Selector | User | Logout                     |
├──────────────────────┬──────────────────────┬──────────────────────────┤
│  OPERATION PANEL     |  CONTEXT PANEL       |  AGENT TRACE PANEL       |
|  (50% width)         |  (25% width)         |  (25% width)             |
|                      |                      |                          |
|  Scanner input       |  HU Card             |  Live step table         |
|  (always focused)    |  Product info        |  Colour-coded rows       |
|  Action buttons      |  Qty + UOM           |  Auto-scroll             |
|  (mode-filtered)     |  Location            |  Debug toggle            |
|  Qty prompt modal    |  Status badge        |  Filter: errors only     |
|                      |  Lineage tree        |  Clear button            |
├──────────────────────┴──────────────────────┴──────────────────────────┤
|  INVENTORY GRID: Location cards showing HU count + status            |
└──────────────────────────────────────────────────────────────────────┘
```

Mode selector (tabs at top of Operation Panel):

- Receiving — actions: Scan, (Create HU placeholder — disabled with tooltip)
- Putaway — actions: Move (requires source + destination)
- Production — actions: Consume (with qty input), Split
- Dispatch — actions: Pick (placeholder — disabled with tooltip)

Interaction rules:

- Action buttons are disabled if: no HU selected, wrong mode for action, HU status incompatible with action
- Disabled buttons show a tooltip explaining WHY
- Scan field auto-focuses on mount and after every action
- Enter key in scan field triggers scan immediately
- Consume action opens an inline quantity input (not a modal overlay) before calling the API
- Move action shows a location selector dropdown populated from `/api/locations`

WebSocket:

- Connect on mount to `ws://localhost:8080/ws`
- On `agent_trace` event: append row to trace panel with fade-in animation
- On `inventory_update` event: refresh the inventory grid and context panel
- Reconnect automatically on disconnect (exponential backoff, max 5s)

Colour system for agent status:

- SUCCESS: green-600 text on green-50 background

- FAILED: red-600 text on red-50 background
- BLOCKED: amber-600 text on amber-50 background

Files required:

```
frontend/src/
  api/client.ts          — axios instance with interceptors, all typed methods
  store/useAppStore.ts   — Zustand: currentHU, currentMode, traceSteps, wsStatus
  hooks/useWebSocket.ts  — WebSocket connection management
  hooks/useScan.ts       — Scan logic: call API, update store, broadcast
  pages/OperatorPage.tsx — Root layout (three panels)
  components/
    OperationPanel.tsx
    ModeSelector.tsx
    ScanInput.tsx
    ActionButtons.tsx
    ContextPanel.tsx
    HUCard.tsx
    LineageTree.tsx
    AgentTracePanel.tsx
    TraceRow.tsx
    InventoryGrid.tsx
    LocationCard.tsx
```

Deliverable: `npm run build` passes with no TypeScript errors. `npm run dev` serves at `http://localhost:5173` connected to live backend.

Agent 5 — Infrastructure Agent

Responsibility: Docker Compose, environment configuration, database migration runner, and development tooling only.

docker-compose.yml (at `/home/opengrace/erplite/`):

```yaml
```

```yaml
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: erplite
      POSTGRES_USER: erplite
      POSTGRES_PASSWORD: erplite_dev_pass
    volumes:
      - pg_data:/var/lib/postgresql/data
      - ./backend/db/migrations:/docker-entrypoint-initdb.d
    ports: ["5432:5432"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U erplite"]
      interval: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile.dev
    environment:
      DB_DSN: postgres://erplite:erplite_dev_pass@postgres:5432/erplite?sslmode=disa
      REDIS_ADDR: redis:6379
      JWT_SECRET: dev_jwt_secret_change_in_prod
      PORT: 8080
    ports: ["8080:8080"]
    volumes:
      - ./backend:/app
    depends_on:
      postgres: { condition: service_healthy }
      redis:    { condition: service_healthy }

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile.dev
    ports: ["5173:5173"]
    volumes:
      - ./frontend:/app
      - /app/node_modules
```

```
    environment:
      VITE_API_BASE: http://localhost:8080
      VITE_WS_BASE: ws://localhost:8080

volumes:
  pg_data:
```

backend/Dockerfile.dev — uses `cosmtrek/air` for hot reload. **frontend/Dockerfile.dev** — uses `vite --host 0.0.0.0`.

Deliverable: `docker compose up --build` starts all services. Backend health check at `GET /health` returns `{"status":"ok"}`. Frontend serves at `http://localhost:5173`.

Agent 6 — Security & Observability Agent

Responsibility: JWT implementation, RBAC middleware, Redis-based idempotency, structured logging, and the agent trace persistence layer.

JWT:

- Access token: 15 minutes, RS256 (generate key pair at startup if not present)
- Refresh token: 7 days, stored in HTTP-only Secure SameSite=Strict cookie
- Claims: `sub` (user public_id), `tenant_id`, `roles[]`, `iat`, `exp`
- No sensitive fields (email, price data) in JWT claims

RBAC middleware:

```go
// Roles: ADMIN, WAREHOUSE_MANAGER, OPERATOR, VIEWER
// Route permissions:
// POST /api/consume, /api/move, /api/split → OPERATOR+
// GET  /api/audit                          → ADMIN only
// GET  /api/inventory, /api/hus/*          → VIEWER+
```

Idempotency (ErrorPrevention Agent):

- On every scan: write `SCAN:{tenant_id}:{barcode}` to Redis with 5-second TTL
- If key exists: return `BLOCKED` with reason `DUPLICATE_SCAN`
- On every consume/move/split: write `OP:{tenant_id}:{hu_public_id}:{operation}` with 30-second TTL

Structured logging:

- Use `log/slog` (stdlib, Go 1.21+)
- Every request logs: `trace_id`, `tenant_id`, `actor_id`, `method`, `path`, `status`, `duration_ms`
- Every agent step logs at DEBUG level with full payload

Audit log:

- Every mutation writes a row to `audit_log` BEFORE and AFTER state
- The `payload_before` and `payload_after` columns must never contain: passwords, JWT tokens, or raw internal IDs (use public_ids only)

Deliverable: All middleware wired into Gin router. Redis connection verified at startup. `go test ./internal/security/...` passes.

NON-NEGOTIABLE CONSTRAINTS (apply to all agents)

1. **No raw SQL string concatenation.** Use sqlc-generated query functions or `pgx` named parameters only.
2. **No direct stock balance updates.** All stock changes are events. Balances are derived.
3. **No HU identity breaks.** A split produces a child HU referencing the parent. The parent retains its ID.
4. **No microservices, no message queues, no Kubernetes.** Single binary, Docker Compose only.
5. **No placeholder/stub implementations.** Every function must do real work. If a feature is future-scope, mark it with a `// TODO(future):` comment and a typed `ErrNotImplemented` return — never a silent no-op.
6. **No V1/V2 branching.** Build the correct model from the first commit.
7. **TypeScript strict mode.** `"strict": true` in tsconfig. No `any` types except in the API client response parser.
8. **Security by default.** No CORS wildcard in production config. `GIN_MODE=release` must produce no debug output.

SUCCESS CRITERIA (system is DONE only when all pass)

Backend:

- [ ] `go build ./...` — zero errors, zero warnings
- [ ] `go test ./...` — all tests pass

- [ ] `go vet ./...` — clean

Frontend:

- [ ] `npm run typecheck` — zero TypeScript errors
- [ ] `npm run build` — builds successfully

Runtime (execute in order after `docker compose up --build`):

1. Open `http://localhost:5173` — login with `admin / admin123`
2. Select **Production** mode
3. Type `HU-1001` in scanner — press Enter
4. Verify Context Panel shows: Product=Fabric Roll, Qty=315 KG, Location=STOR-01, Status=AVAILABLE
5. Verify Agent Trace Panel shows (in order): BarcodeAgent:RESOLVE:SUCCESS → ErrorPreventionAgent:VALIDATE:SUCCESS → InventoryAgent:FETCH:SUCCESS → PricingAgent:RESOLVE_PRICE:SUCCESS → AuditAgent:LOG:SUCCESS
6. Click **Consume** — enter quantity 50 — confirm
7. Verify Context Panel updates: HU-1001 now 265 KG
8. Verify Inventory Grid shows HU-1001 updated
9. Verify Agent Trace shows: ProductionAgent:SPLIT_HU:EXECUTED → InventoryAgent:UPDATE:SUCCESS → LabelingAgent:GENERATE:SUCCESS → AuditAgent:LOG:SUCCESS
10. Scan `HU-1001` again immediately — verify ErrorPreventionAgent:DUPLICATE_SCAN:BLOCKED appears in red
11. Switch to **Putaway** mode — verify Consume/Split buttons are disabled with tooltip "Not available in Putaway mode"
12. Call `GET /api/hus/HU-1001/lineage` — verify response includes parent HU and the newly created child HU with correct quantities

---

FOLDER STRUCTURE (final, enforced)

```
/home/opengrace/erplite/
├── backend/
|   ├── cmd/server/main.go
|   ├── db/
|   |   ├── migrations/001_init.sql
|   |   └── queries/          ← sqlc .sql query files
|   ├── internal/
|   |   ├── agents/
```

```
│   │   │   ├── barcode/
│   │   │   ├── inventory/
│   │   │   ├── warehouse/
│   │   │   ├── production/
│   │   │   ├── labeling/
│   │   │   ├── pricing/
│   │   │   ├── audit/
│   │   │   └── errorprevention/
│   │   ├── api/
│   │   │   ├── handlers/
│   │   │   ├── middleware/
│   │   │   └── router.go
│   │   ├── domain/
│   │   │   ├── models/
│   │   │   ├── events/
│   │   │   └── errors/
│   │   ├── repository/       ← sqlc-generated + wrappers
│   │   ├── service/          ← orchestrates agents
│   │   └── security/
│   ├── Dockerfile.dev
│   ├── go.mod
│   └── sqlc.yaml
├── frontend/
│   ├── src/
│   │   ├── api/client.ts
│   │   ├── store/
│   │   ├── hooks/
│   │   ├── pages/
│   │   └── components/
│   ├── Dockerfile.dev
│   ├── package.json
│   ├── tsconfig.json
│   └── vite.config.ts
└── docker-compose.yml
```